

http://algs4.cs.princeton.edu

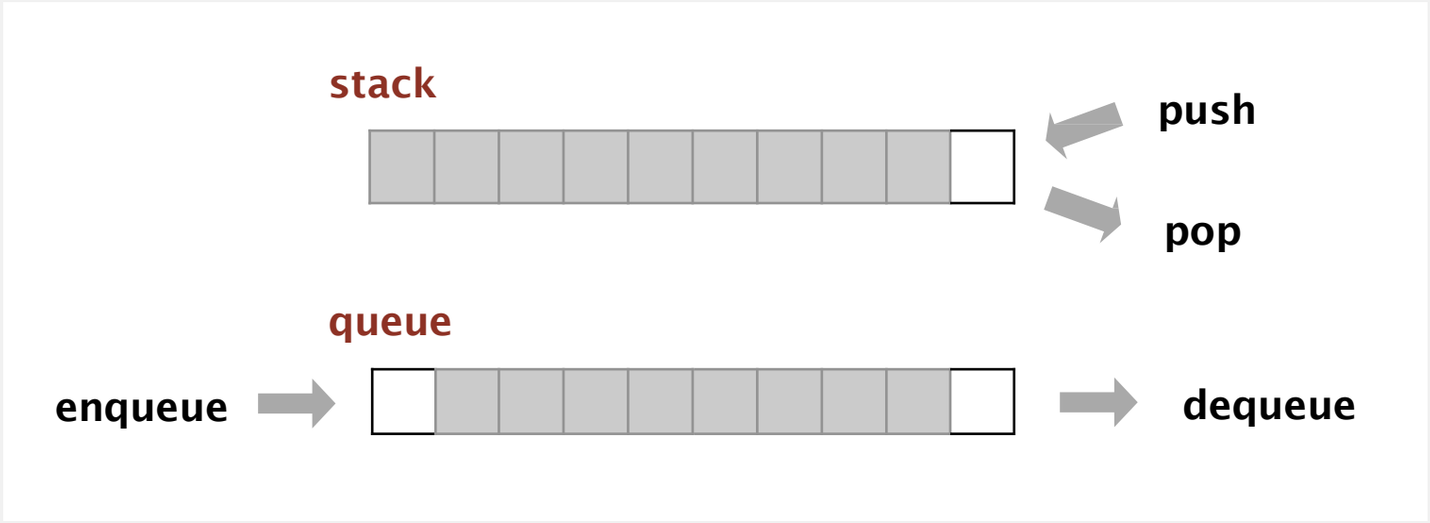
1.3 BAGS, QUEUES, AND STACKS

- *stacks*
- *resizing arrays*
- *queues*
- *generics*
- *iterators*
- *applications*

Stacks and queues

Tipe të të dhënave themelore

- Vlera: collection i objekteve.
- Operacionet: insert, remove, iterate, test if empty.
- Qëllimi i saktë kur ka insert.
- Cilin item duhet me remove?



Stack. Kontrollohet elementi i fundit i shtuar. ← LIFO = "last in first out"

Queue. Kontrollohet elementi më i hershëm i shtuar. ← FIFO = "first in first out"



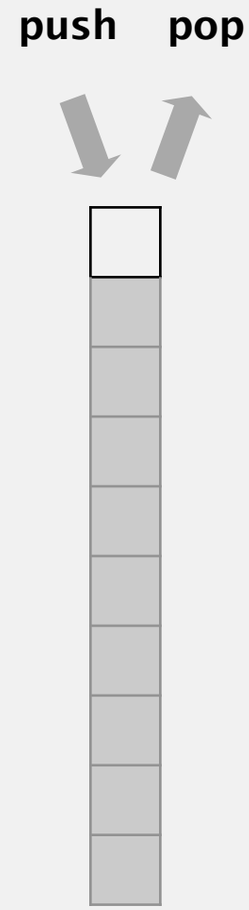
1.3 Bags, Queues, and Stacks

- *stacks*
- *resizing arrays*
- *queues*
- *generics*
- *iterators*
- *applications*

Stack API

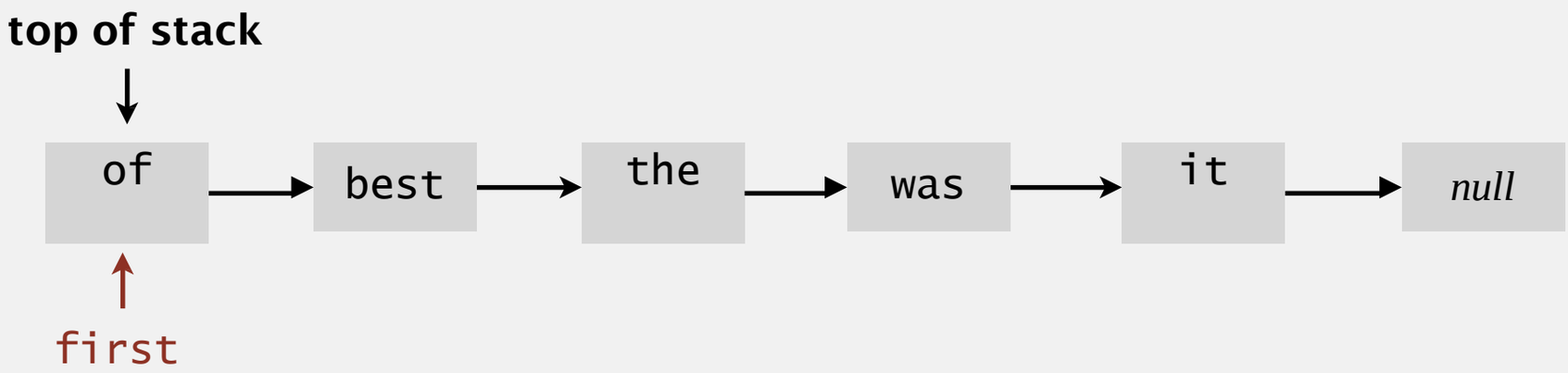
Warmup API. Stack of strings data type.

public class StackOfStrings	
StackOfStrings()	*create an empty stack*
void push(String item)	*insert a new string onto stack*
String pop()	*remove and return the string most recently added*
boolean isEmpty()	*is the stack empty?*
int size()	*number of strings on the stack*



Stack: linked-list implementimi

- Mbahët një pointer `first` tek first node në një singly-linked list.
- Push new item para `first`.
- Pop item prej `first`.



Stack pop: linked-list implementation

inner class

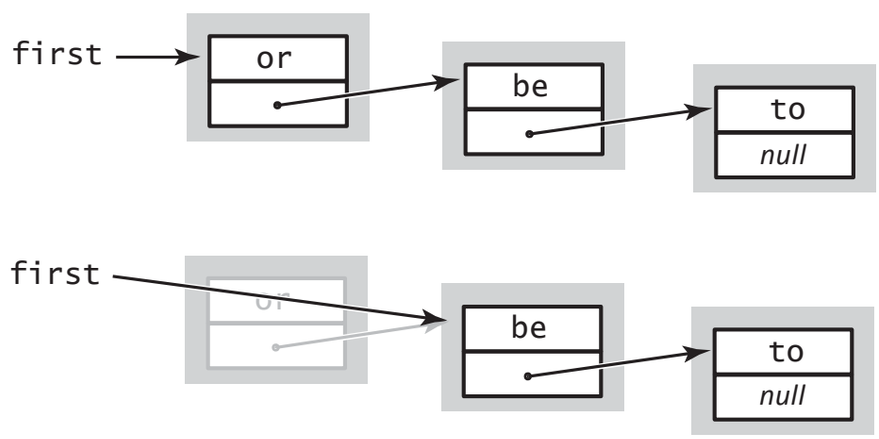
```
private class Node
{
   String item;
   Node next;
}
```

save item to return

```
String item = first.item;
```

delete first node

```
first = first.next;
```



return saved item

```
return item;
```

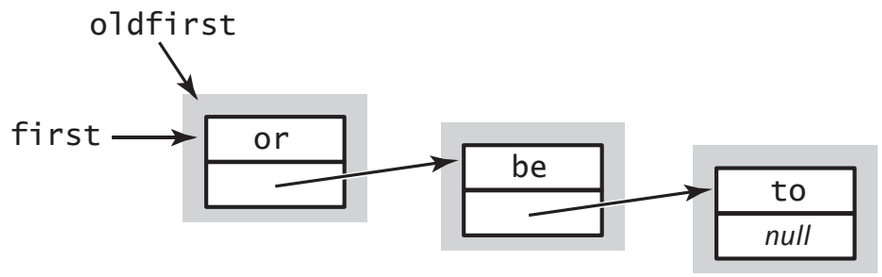
Stack push: linked-list implementimi

inner class

```
private class Node
{
   String item;
   Node next;
}
```

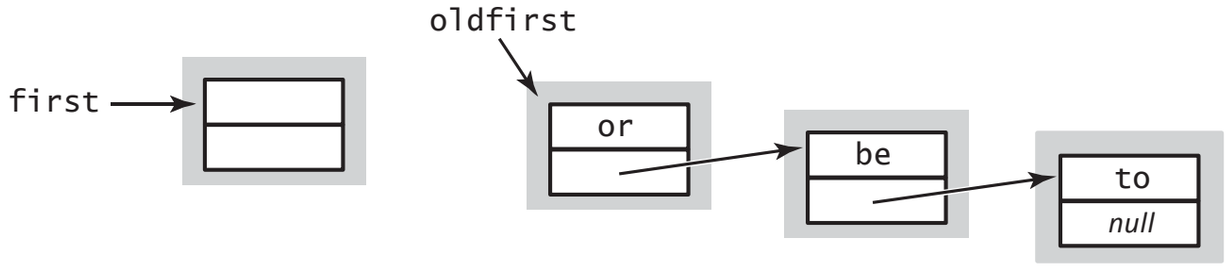
save a link to the list

```
Node oldfirst = first;
```



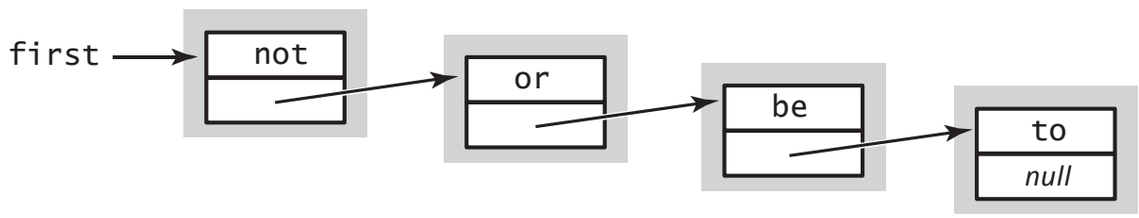
create a new node for the beginning

```
first = new Node();
```



set the instance variables in the new node

```
first.item = "not";
first.next = oldfirst;
```



Stack: linked-list implementimi në Java

```
public class LinkedStackOfStrings
{
   private Node first = null;

   private class Node
   {
      String item;
      Node next;
   }

   public boolean isEmpty()
   {  return first == null;  }

   public void push(String item)
   {
      Node oldfirst = first;
      first = new Node();
      first.item = item;
      first.next = oldfirst;
   }

   public String pop()
   {
      String item = first.item;
      first = first.next;
      return item;
   }
}
```



private inner class

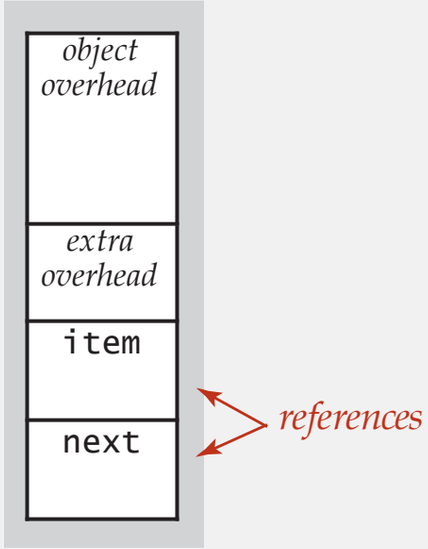
Stack: Performanca e linked-list implementimit

Pohim. Çdo operacion merr kohë konstante në rastin më të keq.

Pohim. Një stack me N items (elemente) përdor $\sim 40\,N$ bytes.

inner class

```
private class Node
{
   String item;
   Node next;
}
```



16 bytes (object overhead)

8 bytes (inner class extra overhead)

8 bytes (reference to String)

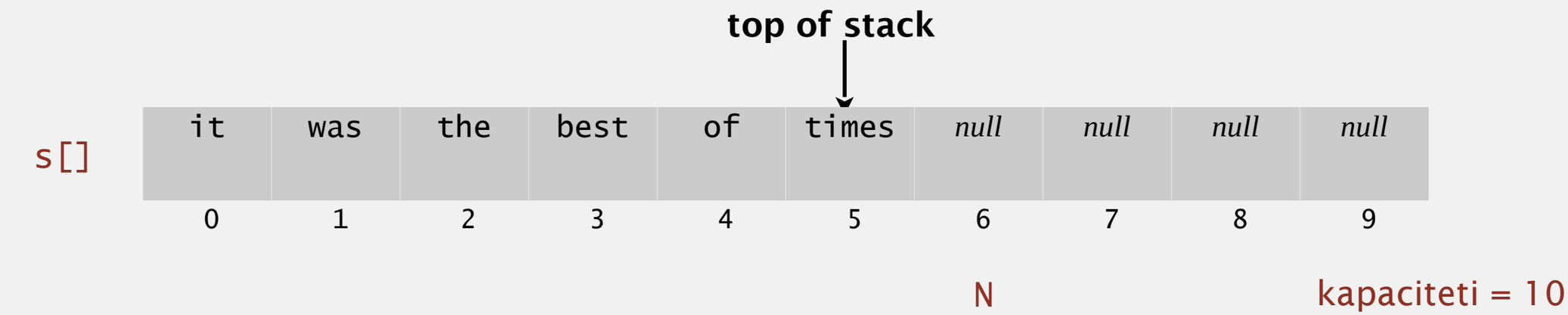
8 bytes (reference to Node)

40 bytes per stack node

Vërejtje. Kjo vlenë për memorien e stack-ut
(jo edhe për memorien e vet string-ave, të cilat i posedon klienti (programi)).

Stack me hapsirë fikse: array implementimi

- Përdoret array s[] për të ruajtur N items në stack.
- `push()`: shto item të ri në `s[N]`.
- `pop()`: merr item prej `s[N-1]`.



Defekt. Stack overflow kur N e kalon kapacitetin.

Stack me hapsirë fikse: array implementimi

```
public class FixedCapacityStackOfStrings
{
   private String[] s;
   private int N = 0;

   public FixedCapacityStackOfStrings(int capacity)
   {  s = new String[capacity];  }

   public boolean isEmpty()
   {  return N == 0;  }

   public void push(String item)
   {  s[N++] = item;  }

   public String pop()
   {  return s[--N];  }
}
```

a cheat
(stay tuned)

Përdoret për të indeksuar në array; pastaj increment N

decrement N;
Pastaj pëdorët për të indeksuar në array

Stack – disa veti

Overflow dhe underflow.

- Underflow: throw exception if pop prej një empty stack.
- Overflow: përdor resizing array për array implementim.

Null item-at. Null item-at lejohen që të insertohen.

Loitering. Mbajtja e një reference të një objekti I cili nuk nevojitet më.

```
public String pop()
{  return s[--N];  }
```

loitering

```
public String pop()
{
   String item = s[--N];
   s[N] = null;
   return item;
}
```

ky version tejkalon "loitering":
garbage collector e kthen memorien për një objekt vetëm nëse nuk ka referenca aktive



1.3 Bags, Queues, and Stacks

- stacks
- resizing arrays
- queues
- generics
- iterators
- applications

Stack: array implementimi

Problem. Klienti nuk mund ta përcakton kapacitetin e array në API!

Q. Si të zvogëlohet/rritet array?

Tentimi i parë.

- `push()`: rritet array `s[]` për `1`.
- `pop()`: zvogëlohet array `s[]` për `1`.

Çmimi i lartë.

- Duhet të kopjohen të gjitha elementet në një array të re, për çdo operacion.
- Array veprimet për të insertuar N *elemente* $= N + (2 + 4 + \ldots + 2(N-1)) \sim N^2$.

s'ka llogari për N të madh

1 array access per push

2(k-1) array veprime të rritet në k (injorohet çmimi i krijimit të array të re)

Sfida. Të sigurohet që ndërrimi i madhësisë së array të ndodhë sa më rrallë.

Stack: array implementimi

Q. Si të rritet një array?

A. Nëse array është plot, krijohet një array dy herë më e madhe, pastaj kopjohen elementet.

"repeated doubling"

```
public ResizingArrayStackOfStrings()
{  s = new String[1];  }

public void push(String item)
{
   if (N == s.length) resize(2 * s.length);
   s[N++] = item;
}

private void resize(int capacity)
{
   String[] copy = new String[capacity];
   for (int i = 0; i < N; i++)
      copy[i] = s[i];
   s = copy;
}
```

Array veprimet për të insertuar N = 2^i elemente.

$$N + (2 + 4 + 8 + \ldots + N) \sim 3N.$$

1 array access per push

k array accesses to double to size k (ignoring cost to create new array)

Stack: array implementimi

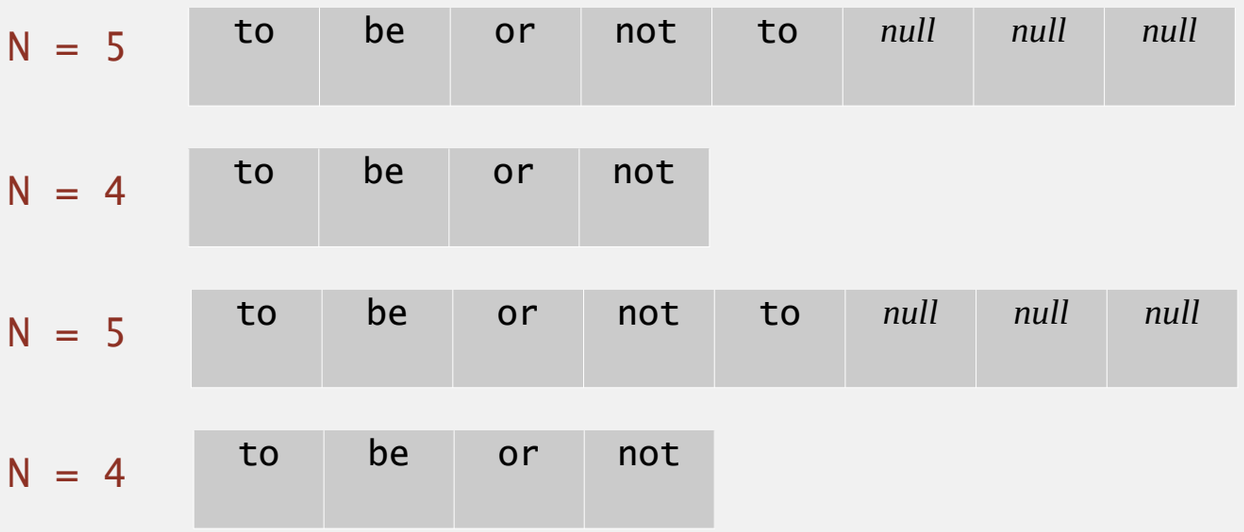
Q. Si të zvogëlohet array?

Tentimi i parë.

- `push()`: dyfishohet madhësia e array `s[]` kur array është plot.
- `pop()`: përgjysmohohet madhësia e array `s[]` kur array është gjysëm e plotë.

Çmimi i lartë.

- push-pop-push-pop-… sekuencë kur array është plot.
- Çdo operacion shpenzon kohë proporcionale me N.



Stack: array implementimi

Q. Si të zvogëlohet array?

Zgjidhja efikase.

- `push()`: dyfishohet madhësia e array `s[]` kur array është plot.
- `pop()`: përgjysmohohet madhësia e array `s[]` kur array është çerek e plotë

```
public String pop()
{
   String item = s[--N];
   s[N] = null;
   if (N > 0 && N == s.length/4) resize(s.length/2);
   return item;
}
```

Kushti. Array është në mes të 25% dhe 100% e mbushur.

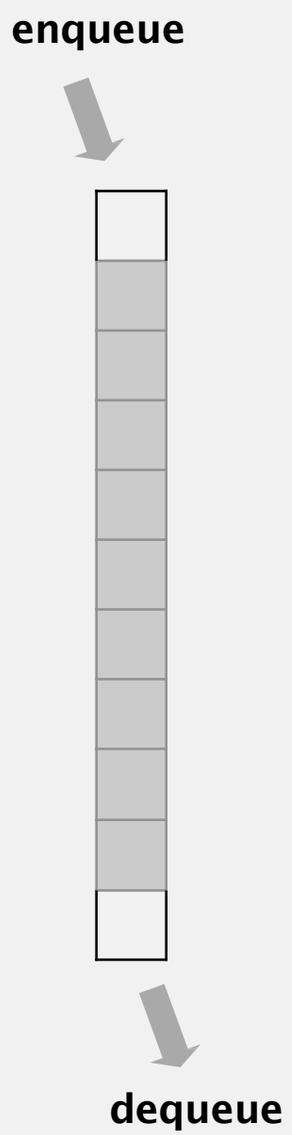


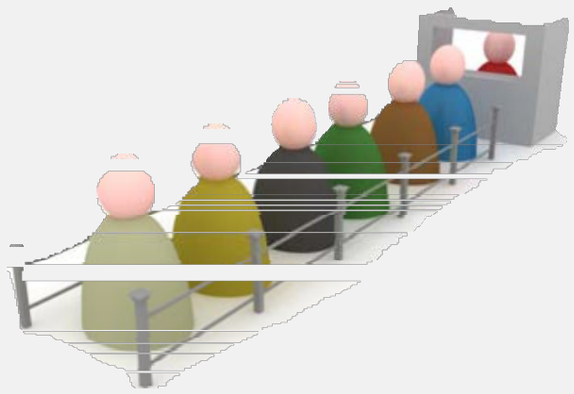
1.3 Bags, Queues, and Stacks

- *stacks*
- *resizing arrays*
- *queues*
- *generics*
- *iterators*
- *applications*

Queue API

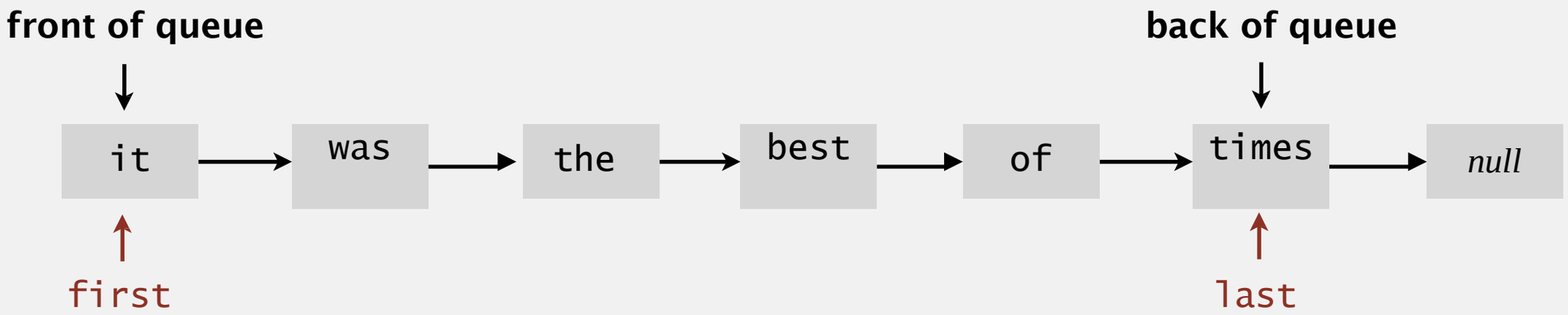
public class QueueOfStrings	
QueueOfStrings()	*create an empty queue*
void enqueue(String item)	*insert a new string onto queue*
String dequeue()	*remove and return the string least recently added*
boolean isEmpty()	*is the queue empty?*
int size()	*number of strings on the queue*





Queue: linked-list implementation

- Mbahet një pointer `first` tek node-i pare në singly-linked list.
- Mbahet një pointer tjetër `last` tek node-i i fundit.
- Dequeue prej `first`.
- Enqueue pas `last`.



Queue dequeue: linked-list implementimi

inner class

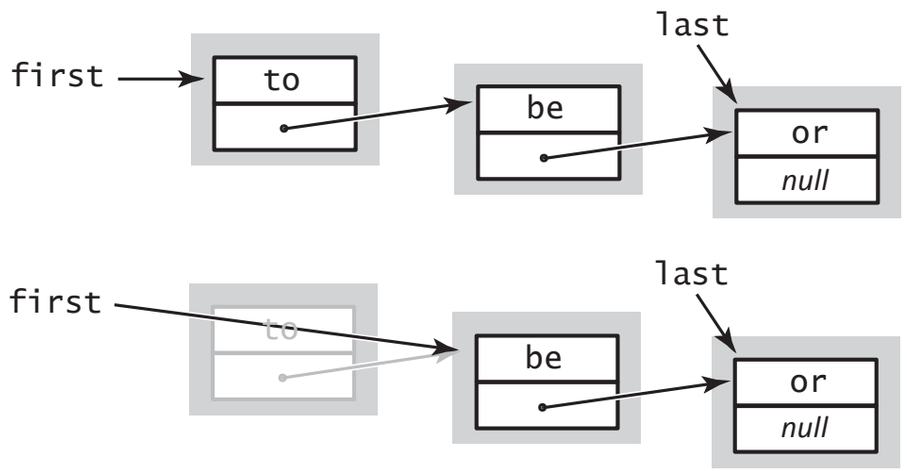
```
private class Node
{
   String item;
   Node next;
}
```

save item to return

```
String item = first.item;
```

delete first node

```
first = first.next;
```



return saved item

```
return item;
```

Vërejte. Kodi identik me linked-list stack `pop()`.

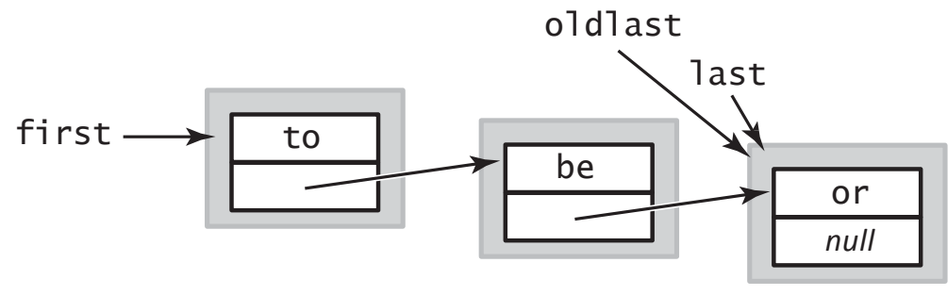
Queue enqueue: linked-list implementimi

inner class

```
private class Node
{
   String item;
   Node next;
}
```

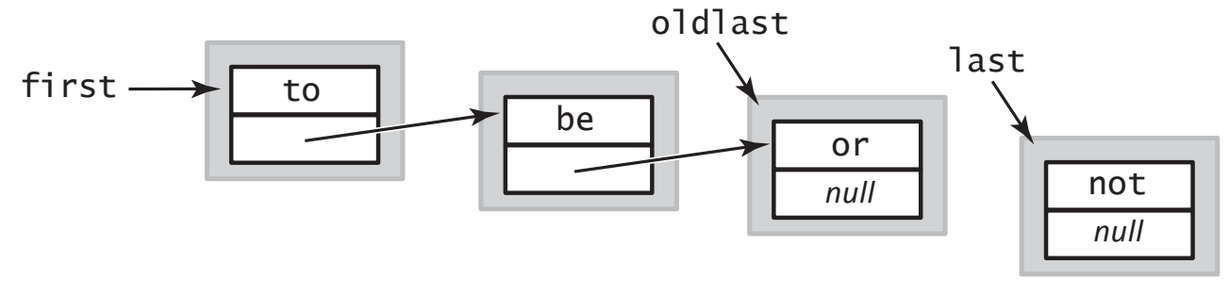
save a link to the last node

```
Node oldlast = last;
```



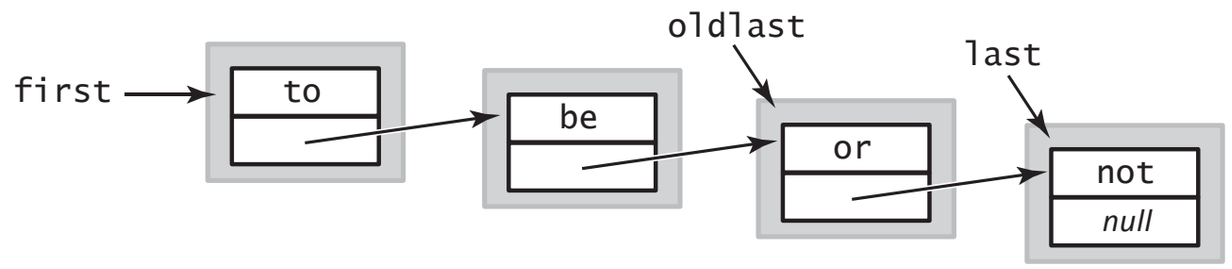
create a new node for the end

```
last = new Node();
last.item = "not";
```



link the new node to the end of the list

```
oldlast.next = last;
```



Queue: linked-list implementimi në Java

```
public class LinkedQueueOfStrings
{
   private Node first, last;

   private class Node
   {  /* same as in LinkedStackOfStrings */  }

   public boolean isEmpty()
   {  return first == null;  }

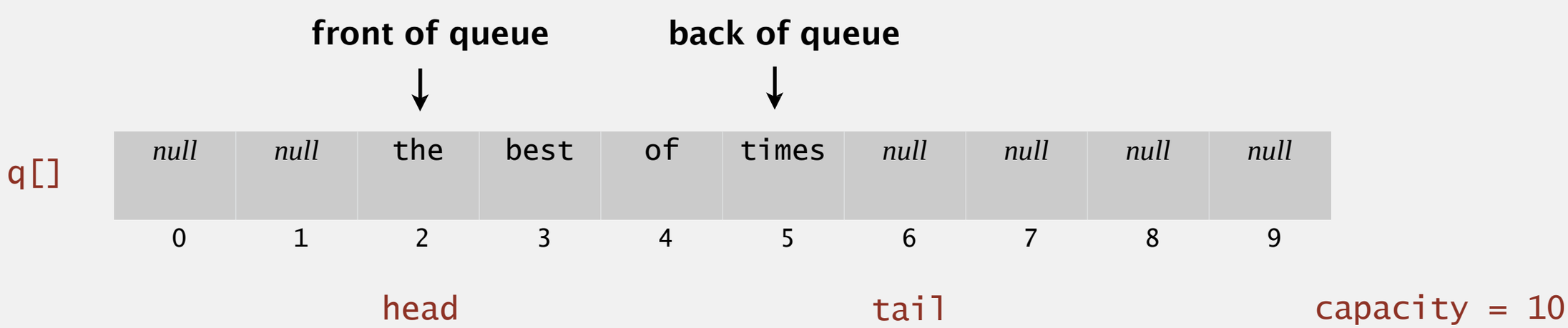
   public void enqueue(String item)
   {
      Node oldlast = last;
      last = new Node();
      last.item = item;
      last.next = null;
      if (isEmpty()) first = last;
      else           oldlast.next = last;
   }

   public String dequeue()
   {
      String item = first.item;
      first       = first.next;
      if (isEmpty()) last = null;
      return item;
   }
}
```

special cases for empty queue

Queue: resizing-array implementation

- Array `q[]` ruan elementet në queue.
- `enqueue()`: shton element të ri në `q[tail]`.
- `dequeue()`: largon element nga `q[head]`.
- Update `head` and `tail` modulo the `capacity`.



Q. Si të rritet/zvogëlohet array?



1.3 Bags, Queues, and Stacks

- *stacks*
- *resizing arrays*
- *queues*
- *generics*
- *iterators*
- *applications*

Parameterized stack

Të implementuar: `StackOfStrings`.

Për t'u implementuar: `StackOfURLs`, `StackOfInts`, `StackOfVans`, ...

Tentimi 1rë. Implementohet një klasë stack për çdo tip.

- Rishkrimi i kodit code është i mërzitshëm dhe me tendencë gabimi (error-prone).
- Mirëmbajtja e kodit cut-dhe-paste gjithashtu është e lodhshme dhe error-prone.

@#$*!



Parameterized stack

Të implementuar : `StackOfStrings`.

Për t'u implementuar : `StackOfURLs`, `StackOfInts`, `StackOfVans`, ...



Tentimi 2të. Implementohet një stack me elemente të tipit `Object`.

- Procesimi është kërkuar te klienti (programi).
- Procesimi është error-prone: run-time error kur tipet nuk pështaten.

```
StackOfObjects s = new StackOfObjects();
Apple  a = new Apple();
Orange b = new Orange();
s.push(a);
s.push(b);
a = (Apple) (s.pop());
```

run-time error

Parameterized stack

Të implementuar : `StackOfStrings`.

Për t'u implementuar : `StackOfURLs`, `StackOfInts`, `StackOfVans`, ...

Tentimi 3të. Java generics.

- Shmanget procesimi në klient (program).
- Zbulohen error-at në compile-time e jo në run-time.

type parameter

```
Stack<Apple> s = new Stack<Apple>();
Apple  a = new Apple();
Orange b = new Orange();
s.push(a);
s.push(b);
a = s.pop();
```

compile-time error

Parimet e punës. Preferohen compile-time errors; shmangen run-time errors.

Generic stack: linked-list implementimi

```
public class LinkedStackOfStrings
{
   private Node first = null;

   private class Node
   {
      String item;
      Node next;
   }

   public boolean isEmpty()
   {  return first == null;  }

   public void push(String item)
   {
      Node oldfirst = first;
      first = new Node();
      first.item = item;
      first.next = oldfirst;
   }

   public String pop()
   {
      String item = first.item;
      first = first.next;
      return item;
   }
}
```

```
public class Stack<Item>
{
   private Node first = null;

   private class Node
   {
      Item item;
      Node next;
   }

   public boolean isEmpty()
   {  return first == null;  }

   public void push(Item item)
   {
      Node oldfirst = first;
      first = new Node();
      first.item = item;
      first.next = oldfirst;
   }

   public Item pop()
   {
      Item item = first.item;
      first = first.next;
      return item;
   }
}
```

generic type name

Generic stack: array implementimi

```
public class FixedCapacityStackOfStrings
{
   private String[] s;
   private int N = 0;

   public ..StackOfStrings(int capacity)
   {  s = new String[capacity];  }

   public boolean isEmpty()
   {  return N == 0;  }

   public void push(String item)
   {  s[N++] = item;  }

   public String pop()
   {  return s[--N];  }
}
```

the way it should be

```
public class FixedCapacityStack<Item>
{
   private Item[] s;
   private int N = 0;

   public FixedCapacityStack(int capacity)
   {  s = new Item[capacity];  }

   public boolean isEmpty()
   {  return N == 0;  }

   public void push(Item item)
   {  s[N++] = item;  }

   public Item pop()
   {  return s[--N];  }
}
```

@#$*! generic array creation not allowed in Java

Generic stack: array implementimi

```
public class FixedCapacityStackOfStrings
{
   private String[] s;
   private int N = 0;

   public ..StackOfStrings(int capacity)
   {  s = new String[capacity];  }

   public boolean isEmpty()
   {  return N == 0;  }

   public void push(String item)
   {  s[N++] = item;  }

   public String pop()
   {  return s[--N];  }
}
```

the way it is

```
public class FixedCapacityStack<Item>
{
   private Item[] s;
   private int N = 0;

   public FixedCapacityStack(int capacity)
   {  s = (Item[]) new Object[capacity]; }

   public boolean isEmpty()
   {  return N == 0;  }

   public void push(Item item)
   {  s[N++] = item;  }

   public Item pop()
   {  return s[--N];  }
}
```

the ugly cast



1.3 Bags, Queues, and Stacks

- *stacks*
- *resizing arrays*
- *queues*
- *generics*
- *iterators*
- *applications*

Iteratorët

Q. What is an `Iterable` ?

A. Has a method that returns an `Iterator`.

java.lang.Iterable interface

```
public interface Iterable<Item>
{
   Iterator<Item> iterator();
}
```

Q. What is an `Iterator` ?

A. Has methods `hasNext()` and `next()`.

java.util.Iterator interface

```
public interface Iterator<Item>
{
   boolean hasNext();
   Item next();
   void remove();
}
```

← optional; use at your own risk

Q. Why make data structures `Iterable` ?

A. Java supports elegant client code.

"foreach" statement (shorthand)

```
for (String s : stack)
   StdOut.println(s);
```

equivalent code (longhand)

```
Iterator<String> i = stack.iterator();
while (i.hasNext())
{
   String s = i.next();
   StdOut.println(s);
}
```

Stack iterator: linked-list implementimi

```
import java.util.Iterator;

public class Stack<Item> implements Iterable<Item>
{
    ...

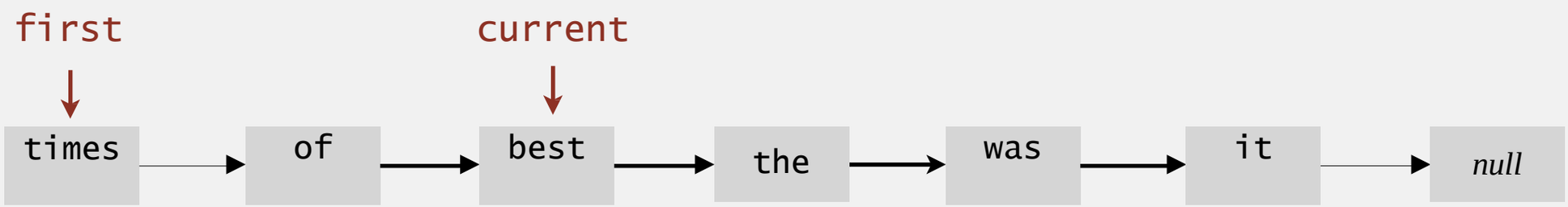
    public Iterator<Item> iterator() { return new ListIterator(); }

    private class ListIterator implements Iterator<Item>
    {
        private Node current = first;

        public boolean hasNext() {  return current != null;  }
        public void remove()     {  /* not supported */      }
        public Item next()
        {
            Item item = current.item;
            current   = current.next;
            return item;
        }
    }
}
```

throw UnsupportedOperationException

throw NoSuchElementException if no more items in iteration



Stack iterator: array implementimi

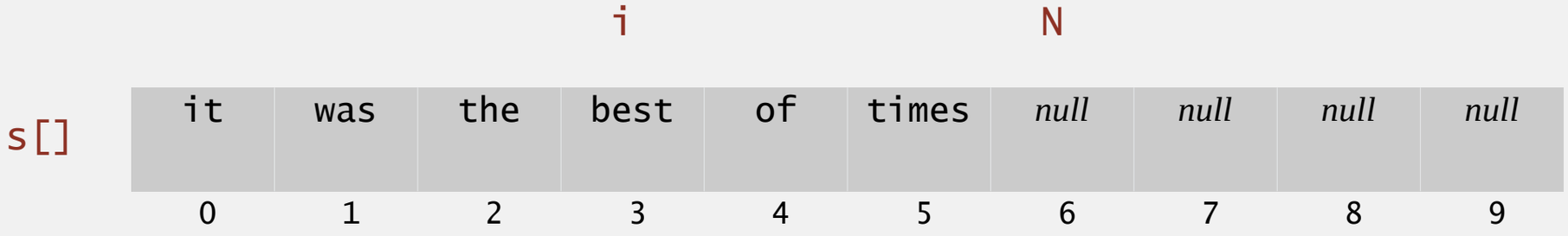
```
import java.util.Iterator;

public class Stack<Item> implements  Iterable<Item>
{
    …

    public Iterator<Item> iterator()
    { return new ReverseArrayIterator(); }

    private class ReverseArrayIterator implements Iterator<Item>
    {
        private int i = N;

        public boolean hasNext() {  return i > 0;          }
        public void remove()     {  /* not supported */   }
        public Item next()       {  return s[--i];         }
    }
}
```





1.3 Bags, Queues, and Stacks

- *stacks*
- *resizing arrays*
- *queues*
- *generics*
- *iterators*
- *applications*

Java collections library

List interface. `java.util.List` is API for an sequence of items.

`public interface List<Item> implements Iterable<Item>`		
	`List()`	*create an empty list*
`boolean`	`isEmpty()`	*is the list empty?*
`int`	`size()`	*number of items*
`void`	`add(Item item)`	*append item to the end*
`Item`	`get(int index)`	*return item at given index*
`Item`	`remove(int index)`	*return and delete item at given index*
`boolean`	`contains(Item item)`	*does the list contain the given item?*
`Iterator<Item>`	`iterator()`	*iterator over all items in the list*
	`...`	

Implementations. `java.util.ArrayList` uses resizing array;
`java.util.LinkedList` uses linked list.



caveat: only some operations are efficient

Java collections library

`java.util.Stack`.

- Supports `push()`, `pop()`, and iteration.
- Extends `java.util.Vector`, which implements `java.util.List` interface from previous slide, including `get()` and `remove()`.
- Bloated and poorly-designed API (why?)

Java 1.3 bug report (June 27, 2001)

> The iterator method on java.util.Stack iterates through a Stack from the bottom up. One would think that it should iterate as if it were popping off the top of the Stack.

status (closed, will not fix)

> It was an incorrect design decision to have Stack extend Vector ("is-a" rather than "has-a"). We sympathize with the submitter but cannot fix this because of compatibility.

Java collections library

`java.util.Stack`.

- Supports `push()`, `pop()`, and iteration.
- Extends `java.util.Vector`, which implements `java.util.List` interface from previous slide, including `get()` and `remove()`.
- Bloated and poorly-designed API (why?)





`java.util.Queue`. An interface, not an implementation of a queue.

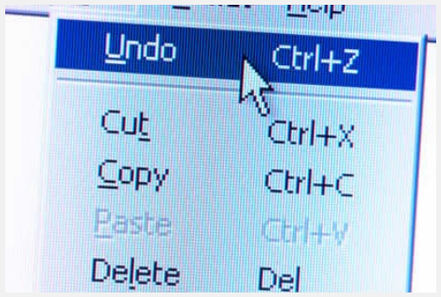
Best practices. Use our implementations of `Stack`, `Queue`, and `Bag`.

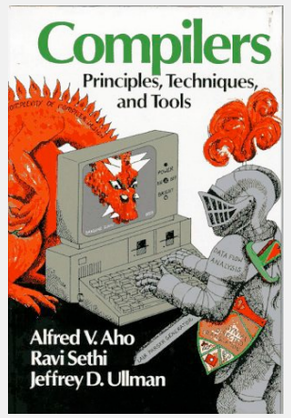
Stack aplikimi

- Parsing në një compiler.
- Java virtual machine.
- Undo në një word processor.
- Back button në një Web browser.
- PostScript language për printerë.
- Implementimi i function calls në një compiler.
- ...









Function calls

Si implementon një compiler një function call.

- Function call: push local environment dhe return address.
- Return: pop return address dhe local environment.

Recursive function. Function që e thërret veten.

p = 216, q = 192

p = 192, q = 24

p = 24, q = 0

gcd (216, 192)

```
static int gcd(int p, int q) {
   if (q == 0) return p;
   else
}
```

gcd (192, 24)

```
static int gcd(int p, int q) {
   if (q == 0) return p;
   else
}
```

gcd (24, 0)

```
static int gcd(int p, int q) {
   if (q == 0) return p;
   else return gcd(q, p % q);
}
```

Llogaritja e shprehjeve aritmetikore

Qëllimi. Llogaritja e infix shprehjes.

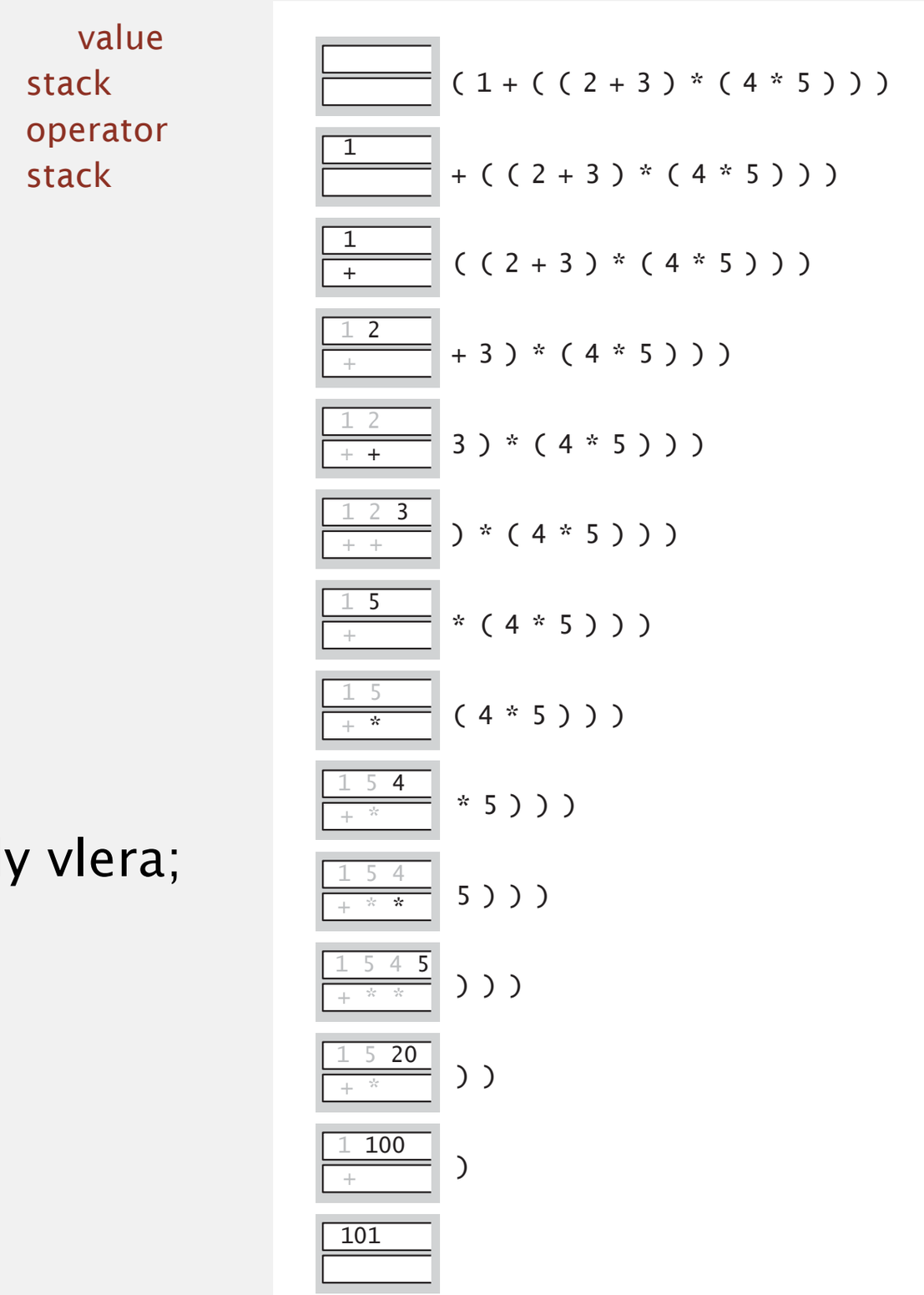
```
( 1 + ( ( 2 + 3 ) * ( 4 * 5 ) ) )
```

operand

operator

Two-stack algoritmi. [E. W. Dijkstra]

- Vlera: push në stack vlera.
- Operator: push në operator stack.
- Kllapa e majtë: injorohet.
- Kllapa e djathtë: pop operatorin dhe dy vlera; push rezultatin në operator stack.



Dijkstra's two-stack algorithm demo



infix expression

(fully parenthesized)

(	1	+	(	(	2	+	3	)	*	(	4	*	5	)	)	)

operand

operator

Llogaritja e shprehjeve aritmetikore

```
public class Evaluate
{
   public static void main(String[] args)
   {
      Stack<String> ops  = new Stack<String>();
      Stack<Double> vals = new Stack<Double>();
      while (!StdIn.isEmpty()) {
         String s = StdIn.readString();
         if      (s.equals("("))                ;
         else if (s.equals("+"))    ops.push(s);
         else if (s.equals("*"))    ops.push(s);
         else if (s.equals(")"))
         {
            String op = ops.pop();
            if      (op.equals("+")) vals.push(vals.pop() + vals.pop());
            else if (op.equals("*")) vals.push(vals.pop() * vals.pop());
         }
         else vals.push(Double.parseDouble(s));
      }
      StdOut.println(vals.pop());
   }
}
```

```
% java Evaluate
( 1 + ( ( 2 + 3 ) * ( 4 * 5 ) ) )
101.0
```

Stack-based gjuhët programuese

Observimi 1. Dijkstra's two-stack algorithm llogaritë vlerat e njejta nëse operatori shfaqet pas dy vlerave.

```
( 1 ( ( 2 3 + ) ( 4 5 * ) * ) + )
```


Jan Lukasiewicz

Obsrvimi 2. Të gjitha kllapat janë të tepërta (redundant)!

```
1 2 3 + 4 5 * * +
```

Përfundim. Postfix or "reverse Polish" notation.

Aplikimet. Postscript, Forth, calculators, Java virtual machine, …